tianracer 底层架构详解

这是一个 tianracer 小车工程代码的底层架构解析文档，我参加了该项目的创新实践课程，选择了工程师 7，要完成底层架构的解读工作，这是工作要求原文

工程师 7：硬件驱动与系统启动模块
负责代码：
- `tianracer_bringup/` 全目录
- `tianracer_jetson/` 全目录
- `tianracer/` 全目录（顶层启动脚本）
- `tianracer_rosinstall`
阅读目标：
1. 阅读 Jetson 平台硬件驱动（电机、舵机、传感器）
2. 梳理节点启动顺序、参数加载与硬件初始化流程
3. 分析硬件通信（串口/ROS 话题）与控制指令闭环
交付物：硬件驱动与启动模块代码解读文档 + 系统启动时序图

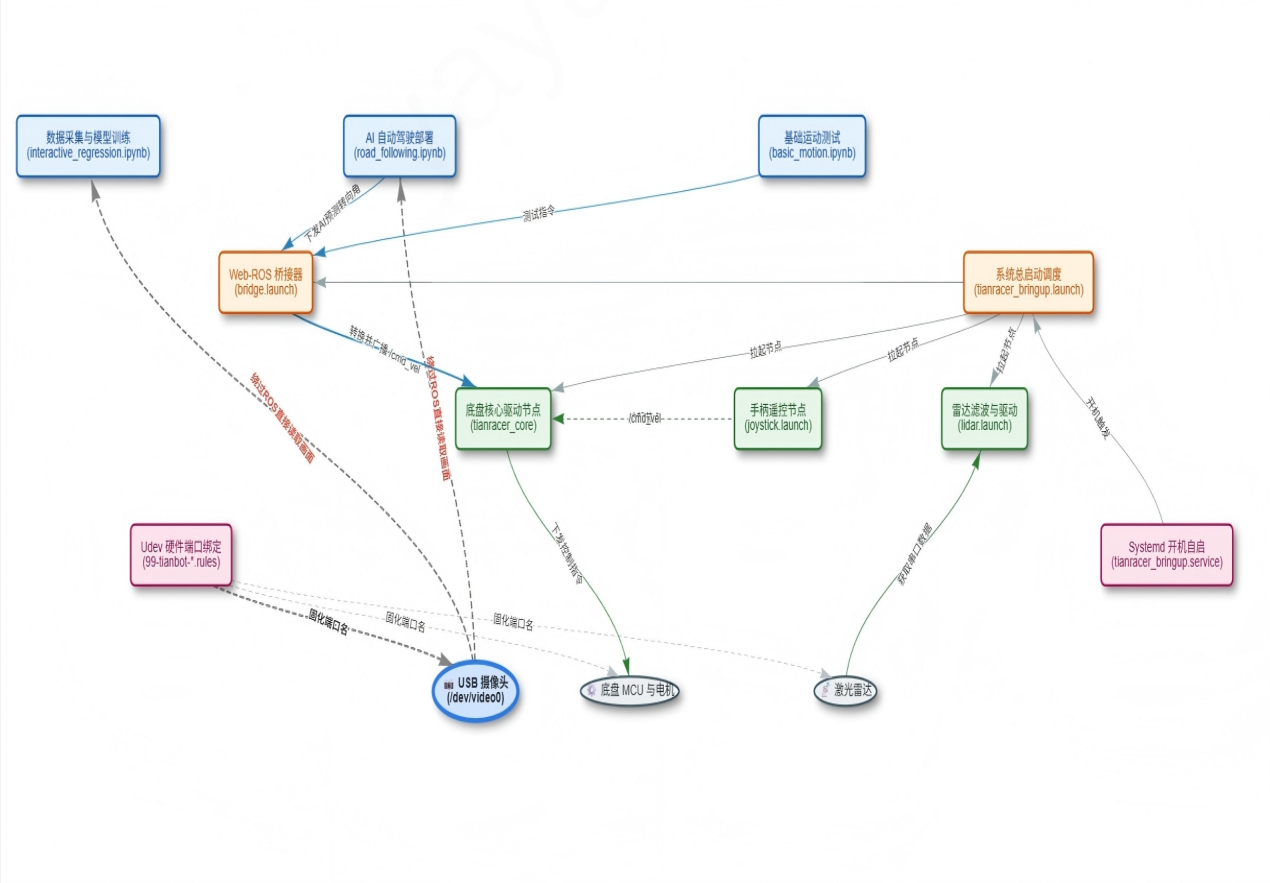
我们分两个主题解析底层架构，分别是系统架构解析和代码结构解析，那么现在开始吧。

一，系统架构解析

首先这是一个 ROS 架构的工程文件夹，会被运行在搭载于 jetson 主板(上位机)内的 ubuntu20.04 系统内,再通过 stm32 单片机（下位机）操作其它硬件设备，如雷达和电机。 ROS 架构讲究一个文件实现一个功能，所以整个文件夹里的结构非常复杂，文件数量也非常多，像我们这样的新手看起来会比较摸不着头脑，懂了 ROS 架构之后再看这个文件结构就会轻松很多,推荐一个比较好的 ROS 架构讲解视频

https://www.bilibili.com/video/BV1De4y1X7r2?vd_source=7c9772c30732ca77c720ac9a26e8a109

我们先不看具体的代码，梳理一下整个小车系统的启动流程和工作逻辑，这是总体的系统流程图



这张图有些乱，我分两部分讲一下流程

1. 系统启动过程：

①ubuntu 系统启动之后，其 systemd 服务自动使文件夹中的“tianracer_bringup.sh”脚本运行。

②“tianracer_bringup.sh”脚本又去执行了“tianracer_bringup.launch”

```
roslaunch tianracer_bringup tianracer_bringup.launch
```

③“tianracer_bringup.launch”又一口气执行了 8 个 launch 文件：

1. “tianracer_tf.launch”开启了 TF（坐标系变换）广播服务。它向全车系统宣告各个传感器的物理安装位置关系，这是后续避障和建图的基础。

2. “joystick_teleop.launch” 驱动了无线手柄，绑定在`/dev/tianbot_joystick`）。拉起`joy_node`（读取硬件摇杆值）和对应的遥控解析节点。开启了人工接管与遥控服务。将人类推动摇杆的物理动作，实时翻译成 ROS 标准的速度控制话题`/cmd_vel`。

3. “lidar.launch”驱动 2D 或 3D 激光雷达，拉起对应厂家的雷达底层驱动节点（如 `rplidarNode`）以及 `laser_filters` 节点。开启了环境二维/三维扫描与自车过滤服务。不仅对外发布原始点云数据 `/scan_raw`，还通过 yaml 文件过滤掉扫到小车自己外壳的废数据，最终输出干净的 `/scan` 话题供算法使用。

4. “rgbd_camera.launch”驱动了高级 3D 深度相机，拉起了相机的 Nodelet Manager 高性能节点群。开启了 三维视觉感知与深度对齐服务。它不仅能输出普通的彩色录像，还能输出带有距离信息的深度图、三维点云数据（Pointcloud），甚至内部的

陀螺仪（IMU）数据。

5. “usb_cam.launch”驱动了普通 2D USB 摄像头,拉起节点 `usb_cam_node`。开启基础平面视觉流媒体服务。以 `mmap` 的高效方式抓取 `/dev/video0` 的画面，发布出 640x480 的标准图像话题。

6. “gps.launch”驱动了外接 GPS 卫星定位模块（遵循 NMEA0183 协议，绑定在 `/dev/tianbot_gps`）。拉起了 `nmea_serial_driver` 节点（节点名重命名为 `tianbot_gps`）。开启了绝对地理坐标定位服务。它以 38400 的波特率读取串口，将卫星信号解析成 ROS 里的经纬度标准话题（NavSatFix）。

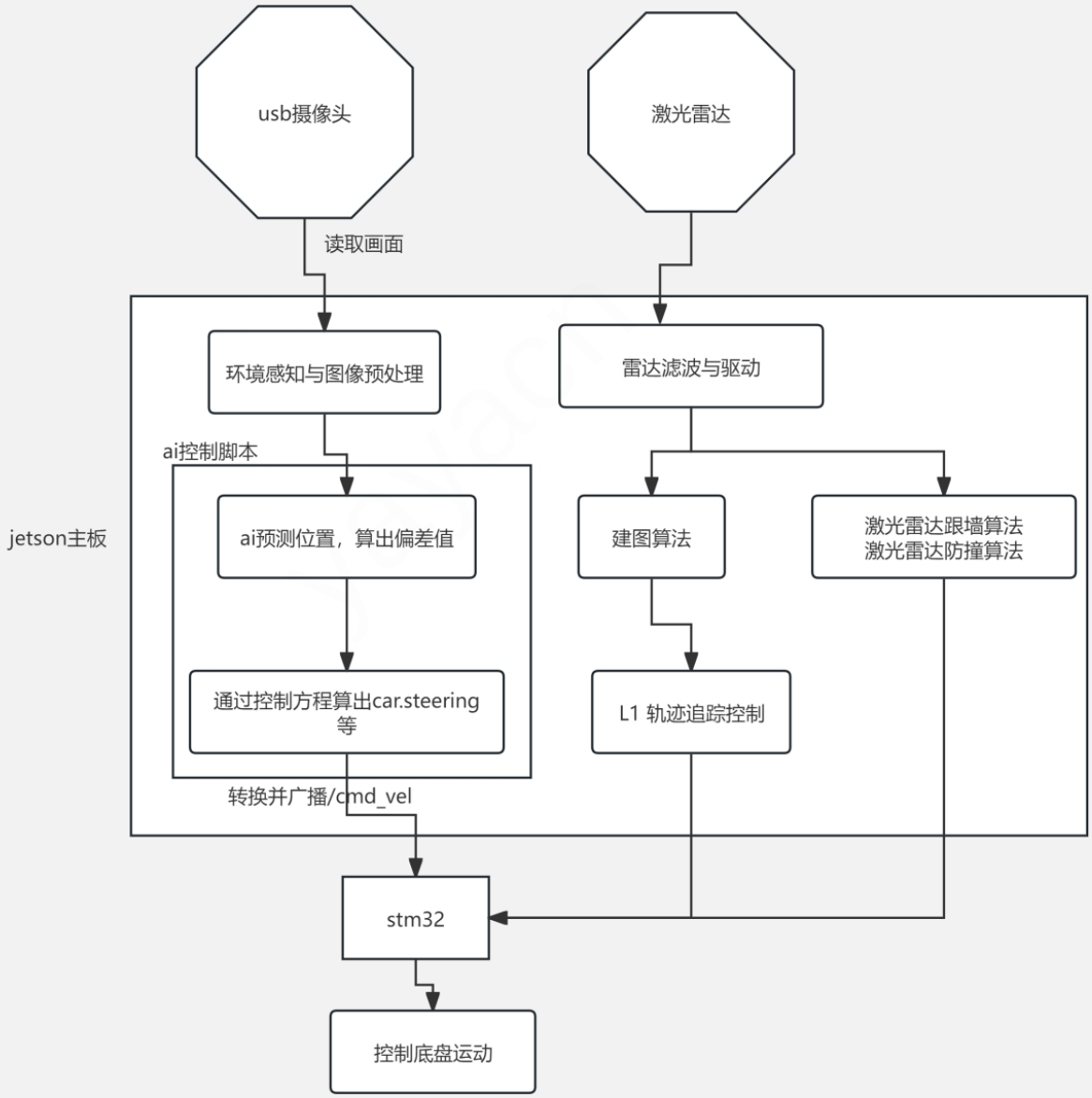
7. “bridge.launch”拉起了`rosbridge_websocket` 和 `tf2_web_republisher` 节点。开启了跨平台 Web Socket 桥接服务。这是全车极其关键的“后门”，它允许 Jetson 里不带 ROS 环境的 Python 脚本、浏览器网页、手机上的 APP，通过局域网直接监听小车的雷达数据，或直接发送速度指令给小车。

8. “tianracer_core.launch”驱动了小车底盘的 STM32 核心控制板及驱动电机（绑定在 `/dev/tianbot_racecar`）。拉起了底盘核心通信节点。开启了 全车物理执行与里程反馈服务。作为下位机 STM32 的“最高长官”，它负责向下把收到的 `/cmd_vel` 翻译成串口十六进制字节流控制电机；向上把 STM32 传回的车轮编码器脉冲数，翻译成小车跑了多远的 `/odom` （里程计）话题。

④见 2

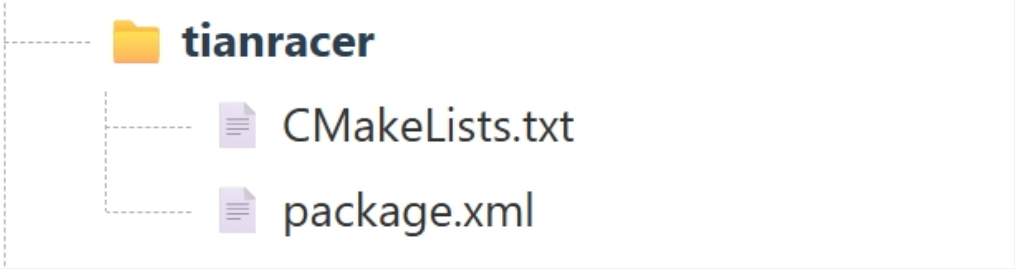
2. 从输入到输出（硬件工作流程）

如图，作一些补充,”road_following_tianracer.ipynb”是纯视觉 ai 控制脚本，“wall_following.py”是激光雷达跟墙算法脚本，建图有 GMapping HectorSLAM Cartographer 三种方式，由于我只考虑自动驾驶场景，所以图中未画出手柄控制的工作模式



二，代码解析

1，tianracer 文件夹



这是一个结构比较简单的文件夹,“顶层启动脚本”其实是不准确的叫法，他其实是一个 ros 架构的顶层元功能包，是一个空壳文件。

-CMakeLists.txt: 声明 tianracer 这个元功能包.并寻找 catkin 构建工具

tianracer > CMakeLists.txt

```
cmake_minimum_required(VERSION 2.8.3)
project(tianracer)
find_package(catkin REQUIRED)
catkin_metapackage()
```

-Package.xml:元功能包的依赖库，按理说这里面应该有各个子功能包的依赖声明，但是这里面好像什么都没有，缺少了依赖声明，只是一个彻头彻尾的空壳文件，猜测可能是在其他的文件夹里写了依赖。

(原代码太长，只截取了一部分)

```
<?xml version="1.0"?>
<package>
  <name>tianracer</name>
  <version>1.0.0</version>
  <description>Tianracer is a meta-package for tianbot racecar</description>

  <!-- One maintainer tag required, multiple allowed, one person per tag -->
  <!-- Example:  -->
  <!-- <maintainer email="jane.doe@example.com">Jane Doe</maintainer> -->
  <maintainer email="tianb03@hotmail.com">Tian Bo</maintainer>
```

2.tianracer.rosinstall 文件

```
tianracer.rosinstall
# THIS IS AN AUTOGENERATED FILE, LAST GENERATED USING wstool ON 2018-06-01
- git:
    local-name: rplidar_ros
    uri: https://github.com/Slamtec/rplidar_ros.git
- git:
    local-name: realsense2_camera
    uri: https://github.com/intel-ros/realsense.git
    version: 2.2.3
- git:
    local-name: rf2o_laser_odometry
    uri: https://github.com/tianb03/rf2o_laser_odometry.git
- git:
    local-name: tianracer
    uri: https://github.com/tianbot/tianracer.git
```

看到这个文件，我们就懂了为什么之前的 package 里没有写任何依赖但是系统仍能跑通。系统采用了源码级依赖管理方案，通过该 rosinstall 文件，配合 wstool 工具，在环境构建初期将核心硬件的外部驱动以及核心算法以 Git 源码克隆的方式强绑定到本地工作空间，也就不需要再在 package 里写依赖了。

这四个 git:

rplidar_ros：激光雷达的官方 ROS 驱动。

realsense2_camera：Intel 深度相机的官方驱动。

rf2o_laser_odometry：开源激光里程计算法。

tianracer：主体源码仓库。

3，tianracer_bringup 文件夹

这是系统启动加载硬件的文件夹，结构比较复杂，一部分是配置文件，

一部分是脚本文件，这个图应该还是挺好理解的

4,tianracer_core

这是和 stm32 进行串口通讯的核心文件夹，

protocol.h：通信协议的“密码本”。定义了上位机（Jetson）与下位机（STM32）之间数据包的帧头（如 `0xAA55`）、指令类型以及数据结构的格式。

serial.h：串口通信的“接口清单”。声明了如何打开 USB 串口、如何配置波特率、以及如何发送和接收字节流的函数。

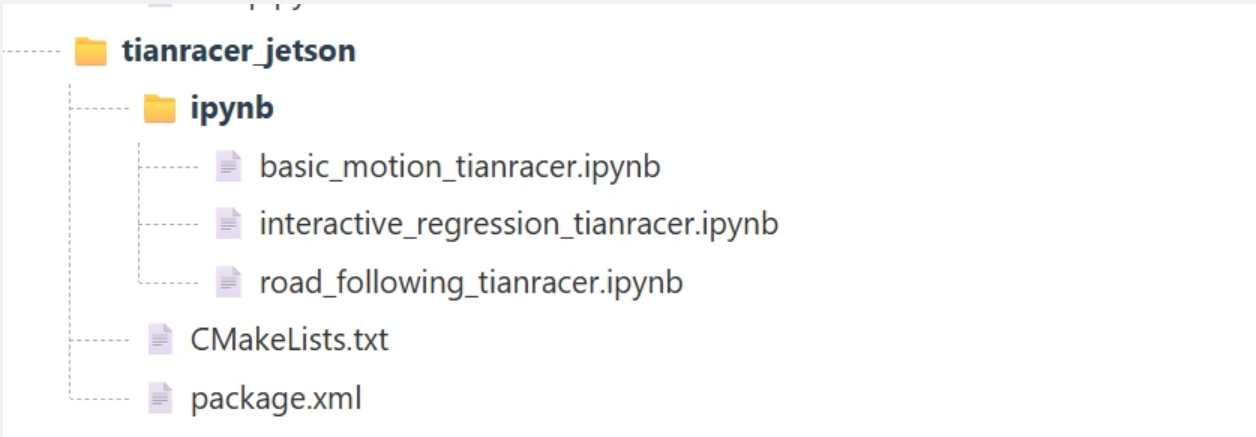
tianboard.h：核心驱动类的“设计图纸”。声明了 ROS 节点该订阅哪些话题（如速度）、发布哪些话题（如里程计和 IMU），以及定时器和转换函数的结构。

tianracer_core.launch：底盘驱动的总启动开关。它一键拉起底盘串口通信节

点，并同时启动 EKF 节点（用来融合计算更准的小车位置）。

main.cpp：程序的入口。负责初始化 ROS 节点，创建 `Tianboard` 对象，并以设定的频率（如 10Hz）死循环维持整个驱动程序的运行。

5. tianracer_jetson



这个文件夹存放了一些 ai 交互脚本

basic_motion_tianracer.ipynb：基础运动调测脚本。提供了一个在网页端直接测试小车油门和转向比例映射关系的工具。

interactive_regression_tianracer.ipynb：数据采集与模型训练脚本。通过摄像头实时抓取画面，供人工在网页上点击标注赛道中心点，并一键启动 ResNet18 神经网络训练。

road_following_tianracer.ipynb：AI 自动驾驶部署脚本。加载训练好并经过 TensorRT 优化的模型，让小车根据摄像头画面自主计算方向盘角度，实现端到端的纯视觉循迹驾驶。

一些话：

我之前玩过单片机，参加过人型足球机器人的比赛，当时学了机器人的一些基础操作，如何改代码参数 编译运行 连接比赛机 ，这都是比较偏顶层应用的操作，我学到了很多，但对底层的一些东西不太了解，我想知道机器的底层究竟发生了什么，我想我之所以对这个感兴趣，和我一年前对物理感兴趣是出于同样目的的，也就是我想知道一个宏观的现象在底层的物理世界中究竟是发生了什么，而不是只知道

这一个宏观的现象存在，因此我选择了工程师 7，去了解底层的架构，虽然没有完全懂，我感觉收获还是挺多的，以后看工程文件夹就比较清晰一些，后续开发一些功能也需要 ros 的知识，不过恕我直言 ros1 还是太老旧了，后续我还是学现在常用的 ros2 架构吧。

这个文档内容总体由我手打（当然借鉴了 ai 的一些分析），总流程图是拿 gemini 生成的，第二个流程图是我做的（按照我的理解做的，但是我对那部分文件不是特别了解，所以不知道准不准确），文件结构图是我拿脚本把整个文件夹里的文件名变成一个 html 文件然后放浏览器上渲染的，然后又加了一些注释

yayacn